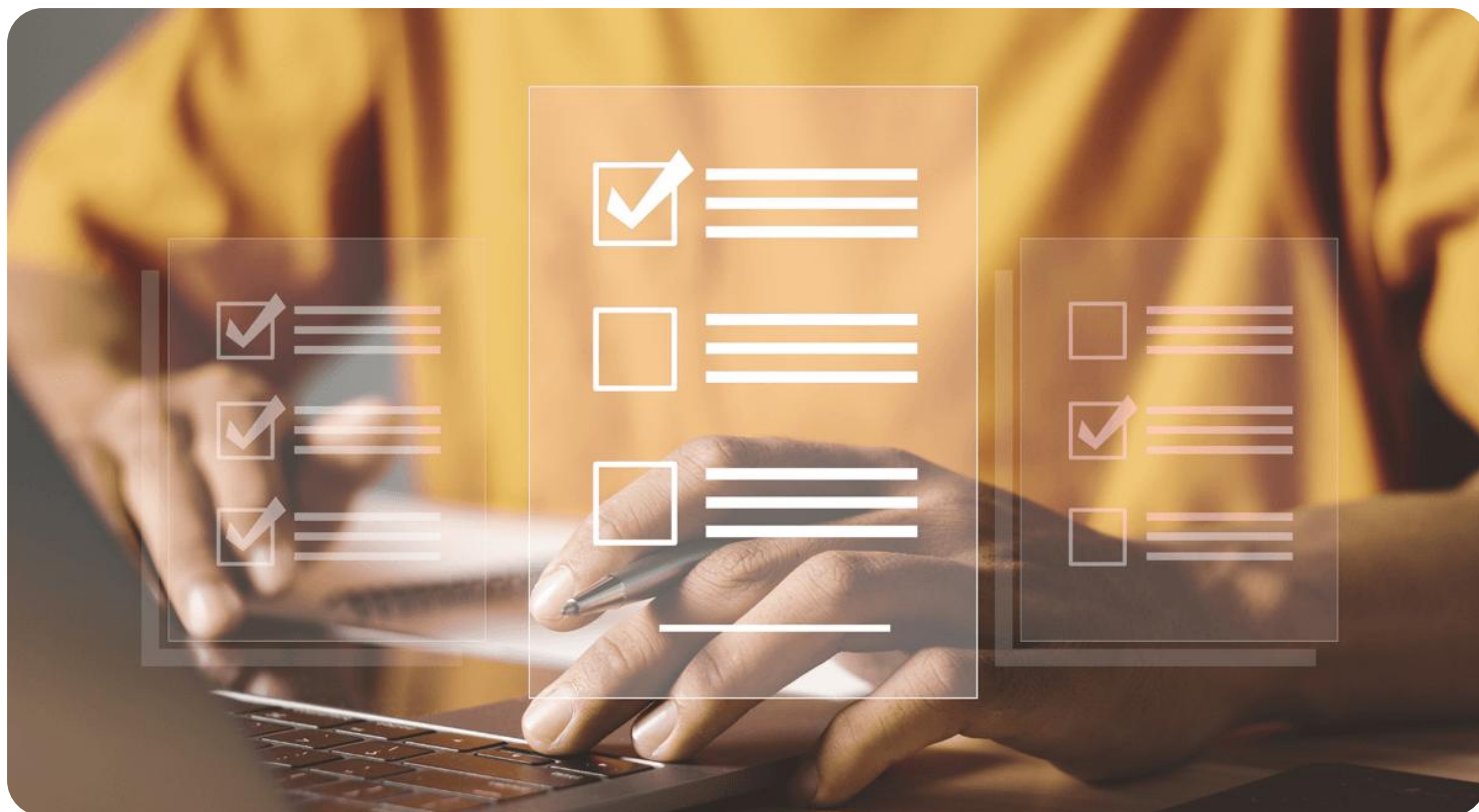




Semana 8

Desarrollo Backend III

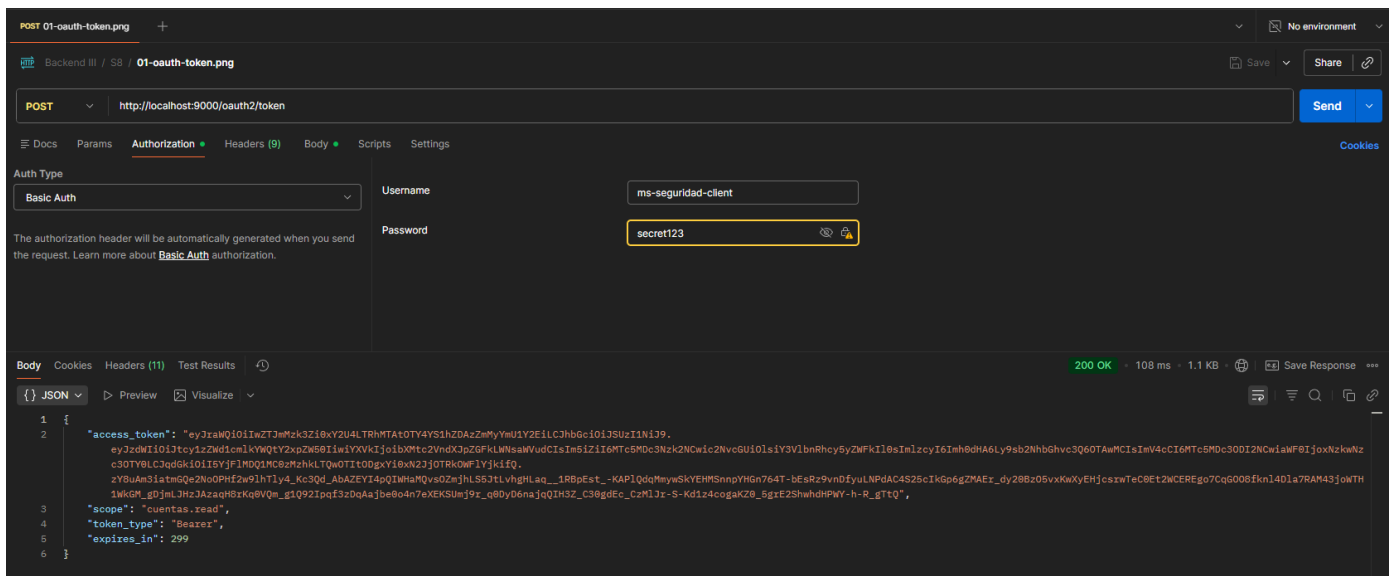
(PBY2203)

Nombre estudiante:	Pedro Andrés Breit Lira
Carrera: Analista Programado Computacional	
Profesor: Alonso Esteban Castillo Pizarro	Fecha: 04/10/26

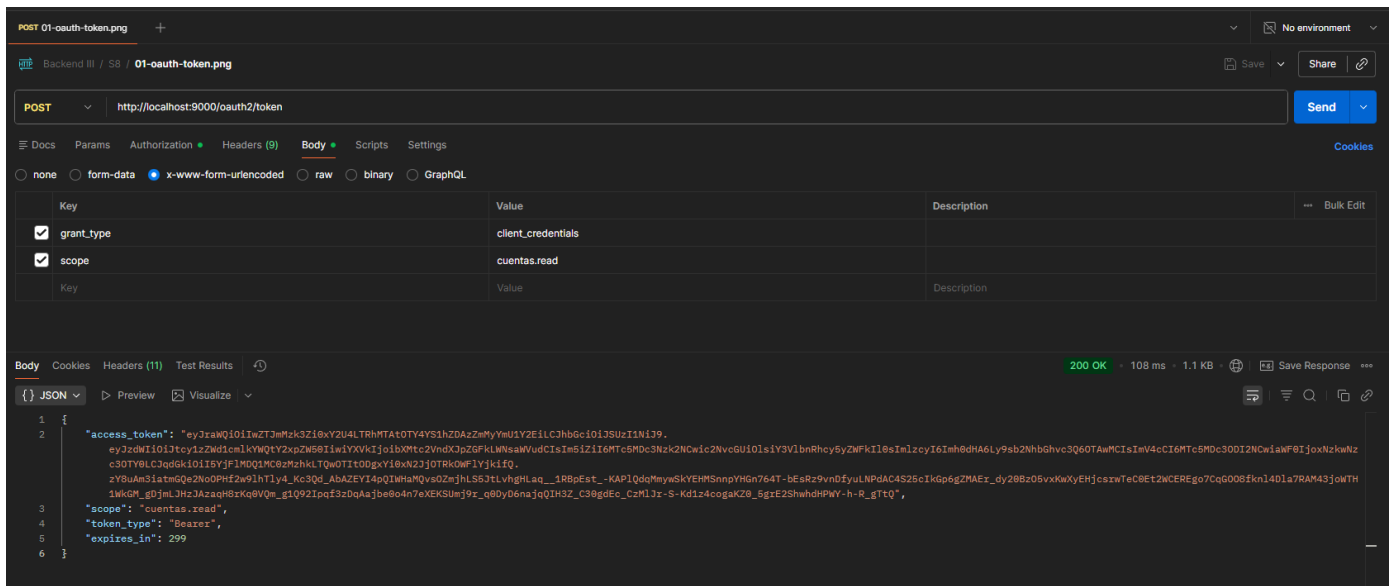
OAuth2 - Obtención de token	3
OAuth2 - Acceso autorizado	4
OAuth2 - Acceso sin token.....	5
OAuth2 - Control de scopes.....	6
Apache Kafka - Solicitud de retiro PENDIENTE	7
Apache Kafka - Resultado APROBADO	8
Apache Kafka - Retiro rechazado	9
Eureka Server - Descubrimiento de servicios	10
Spring Cloud Config Server	11
Resilience4j - Fallback ante caída de Banco Central.....	12

OAuth2 - Obtención de token

El Authorization Server se ejecuta en el puerto 9000 y utiliza el flujo client_credentials. El cliente de desarrollo ms-seguridad-client se autentica mediante Basic Auth y solicita el scope cuentas.read.



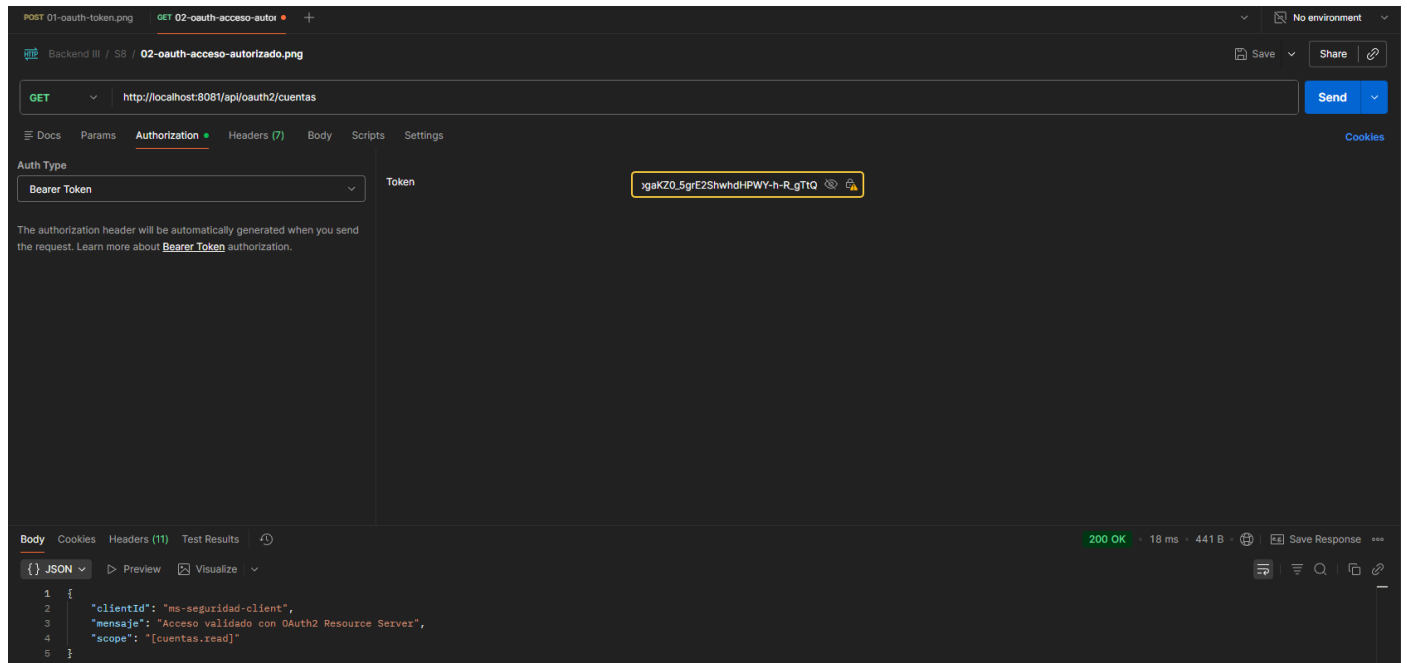
Configuración de Basic Auth en Postman para el cliente OAuth2. Se utiliza la contraseña de desarrollo secret123.



Solicitud POST /oauth2/token con grant_type=client_credentials y scope=cuentas.read. El servidor responde HTTP 200 y entrega un access token.

OAuth2 - Acceso autorizado

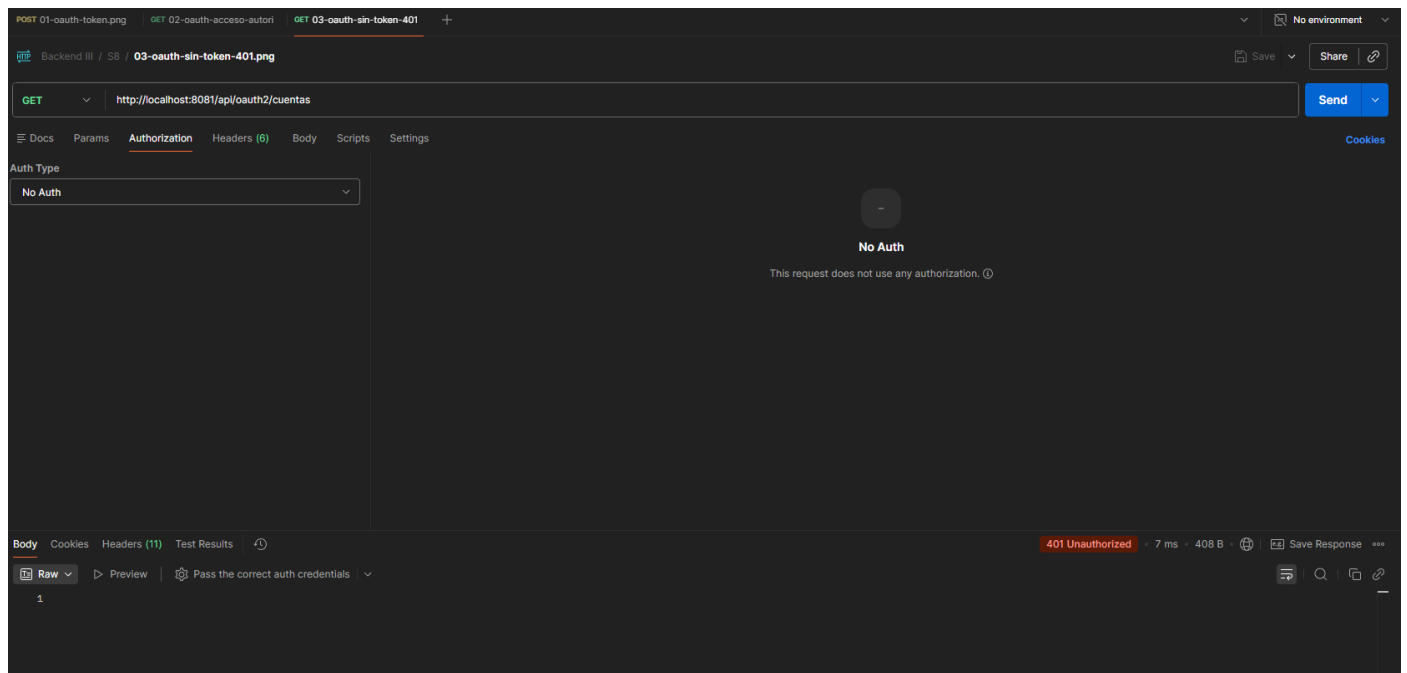
Con el Bearer Token emitido por el Authorization Server se accede al recurso protegido GET /api/oauth2/cuentas del Banco Central. La respuesta HTTP 200 confirma la validación del token y del scope cuentas.read.



Acceso autorizado al recurso protegido del Banco Central utilizando un token OAuth2 válido.

OAuth2 - Acceso sin token

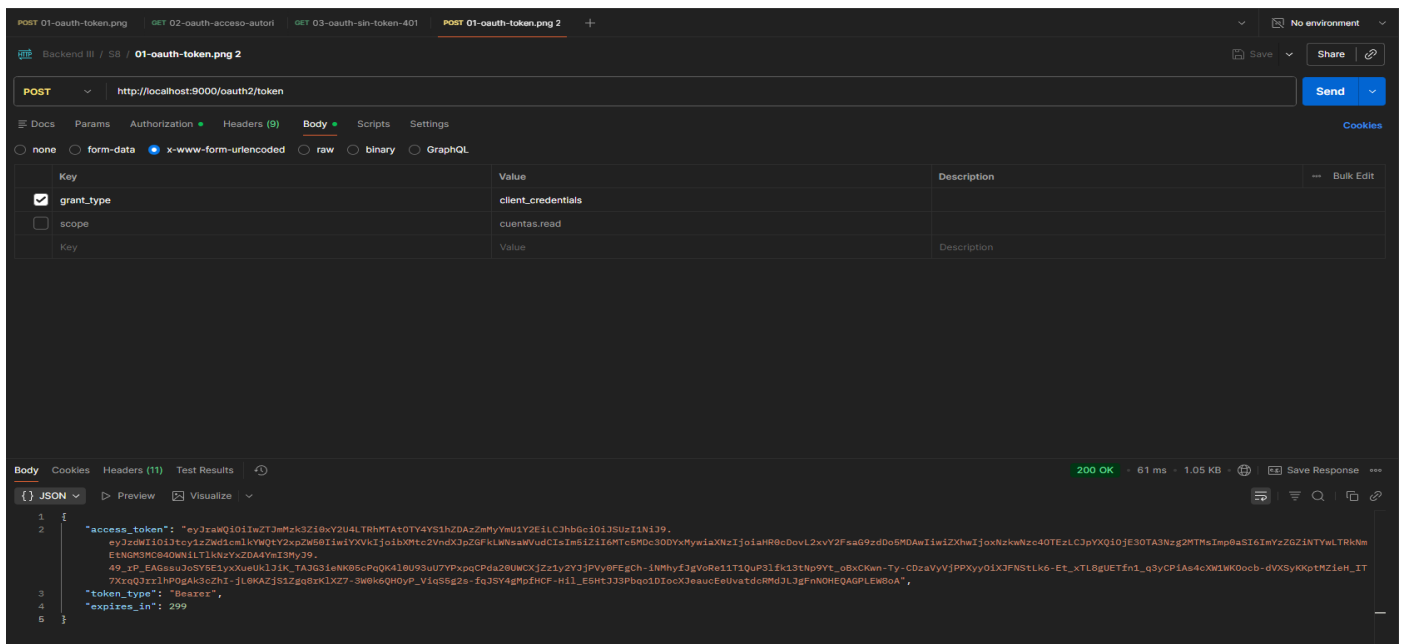
La misma operación se ejecuta sin credenciales. Spring Security rechaza la solicitud con HTTP 401 Unauthorized, comprobando que el recurso no puede ser consumido de forma anónima.



Respuesta 401 Unauthorized al intentar acceder al recurso protegido sin Bearer Token.

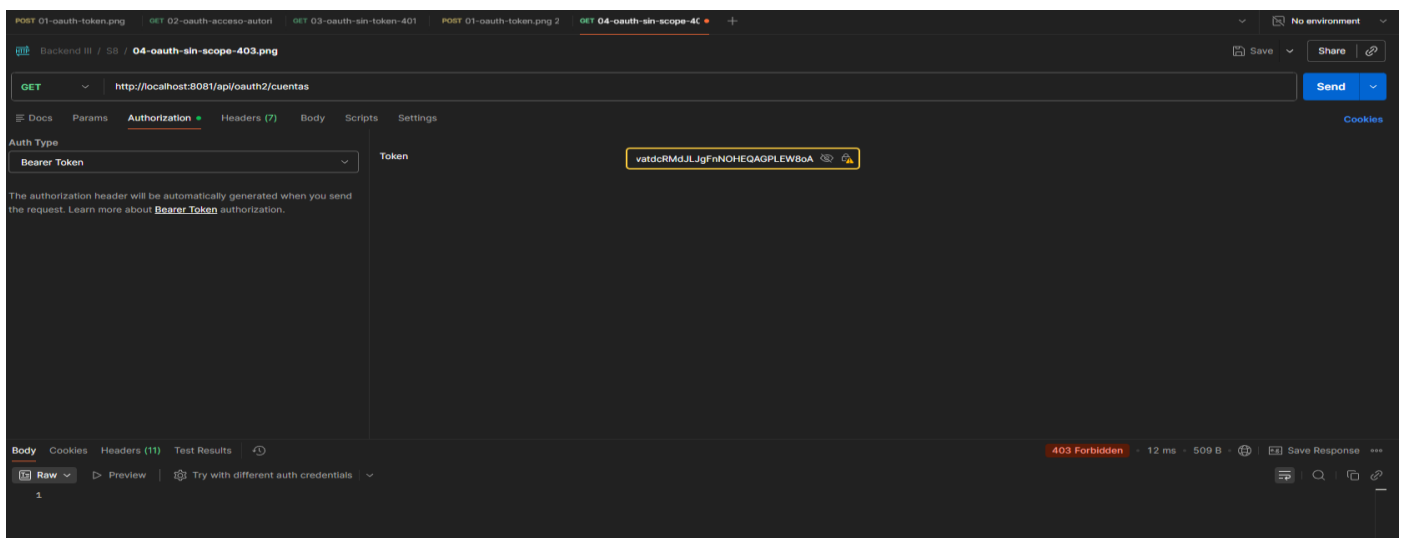
OAuth2 - Control de scopes

Para validar la autorización se genera un token sin solicitar el scope cuentas.read. El token es emitido, pero no contiene el permiso requerido para consumir el endpoint protegido.



Token OAuth2 generado sin el scope requerido cuentas.read.

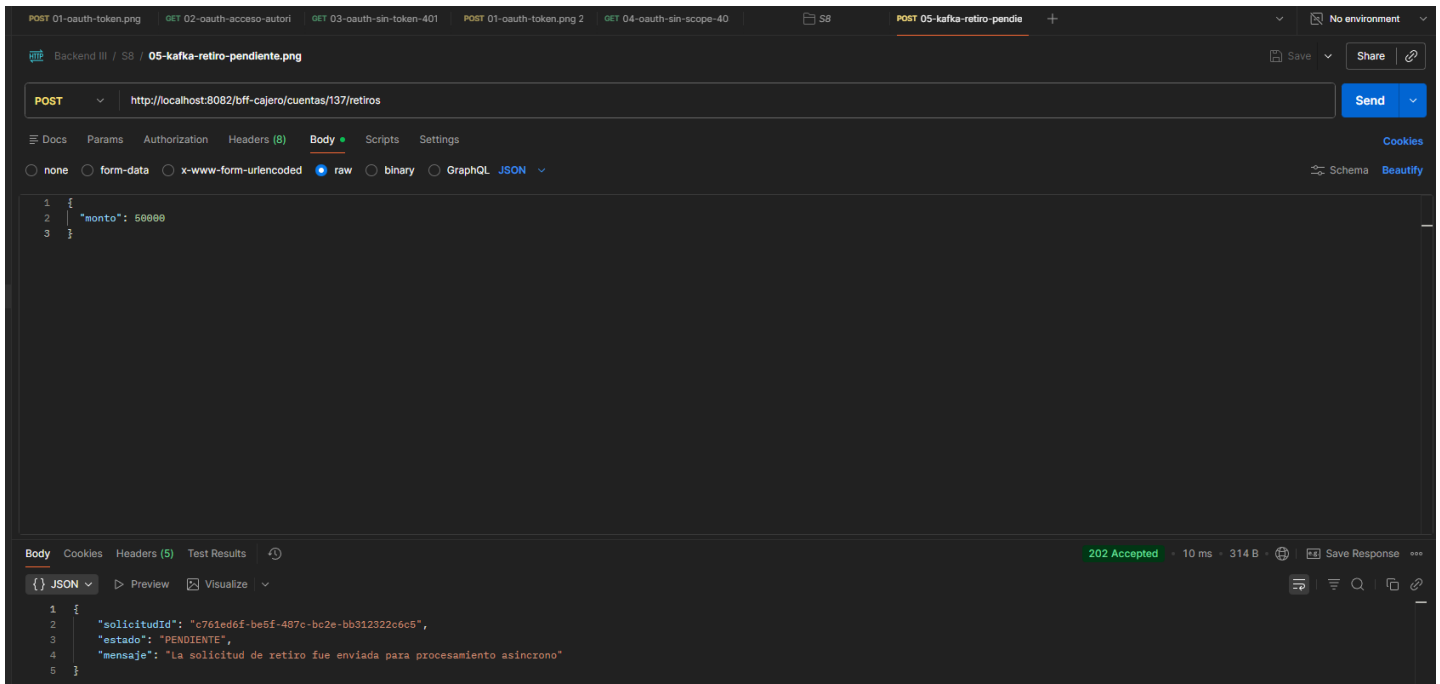
Al utilizar este token en el recurso protegido, el Resource Server responde HTTP 403 Forbidden. Esto evidencia la diferencia entre autenticación del cliente y autorización por scopes.



Respuesta 403 Forbidden al utilizar un token autenticado pero sin el scope requerido.

Apache Kafka - Solicitud de retiro PENDIENTE

El BFF Cajeros recibe una solicitud POST de retiro por \$50.000 y responde HTTP 202 Accepted con un solicitudId y estado PENDIENTE. La operación se publica de forma asíncrona en el tópico retiros-solicitados, sin bloquear al cliente mientras Banco Central procesa el evento.

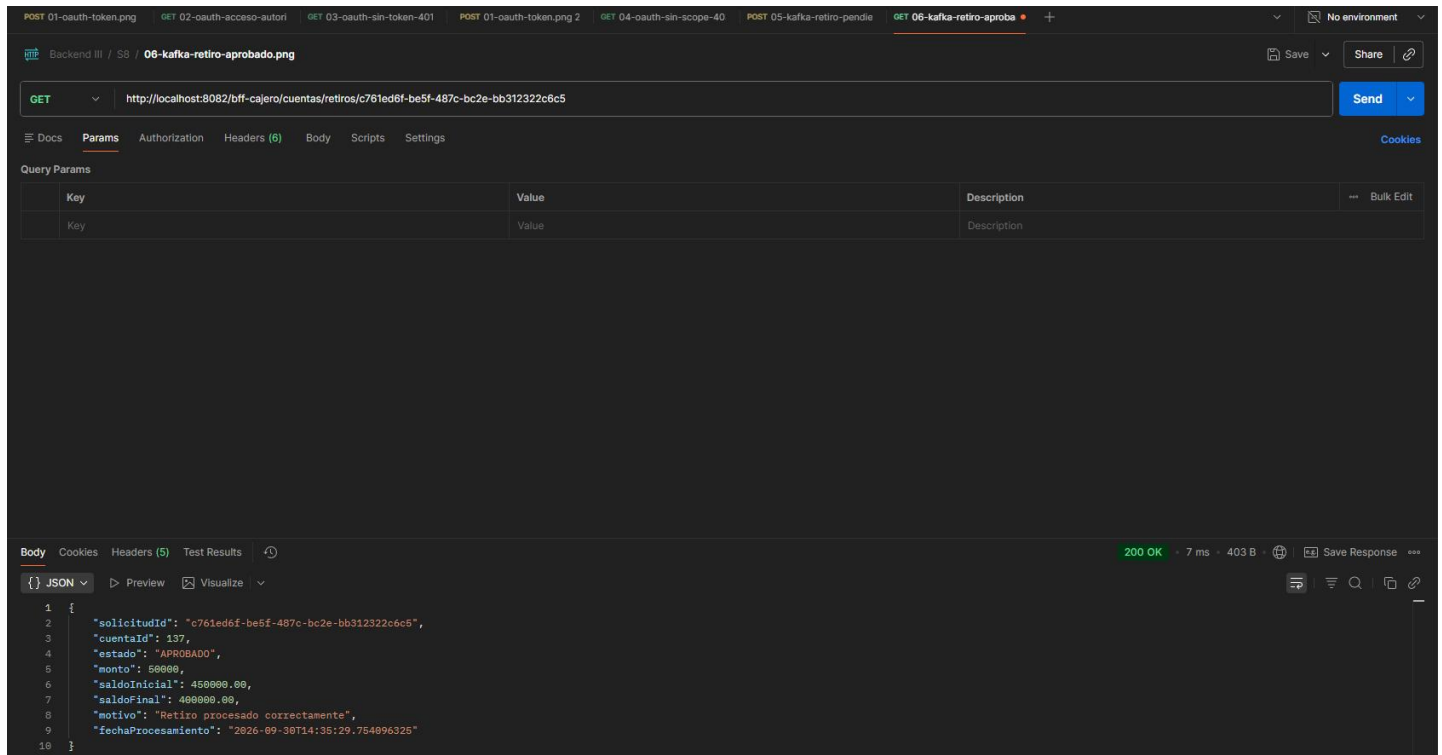


Solicitud de retiro aceptada de forma asíncrona por BFF Cajeros. El estado inicial PENDIENTE confirma que el procesamiento continúa mediante Kafka.

Los tópicos utilizados para el flujo son retiros-solicitados, retiros-aprobados y retiros-rechazados, cada uno configurado con tres particiones para permitir procesamiento concurrente.

Apache Kafka - Resultado APROBADO

Luego del procesamiento en Banco Central, el resultado es publicado y consumido por el BFF Cajeros. La consulta del solicitudId retorna estado APROBADO y muestra la actualización de saldo desde \$450.000 a \$400.000.



The screenshot shows a REST client interface with a list of requests at the top. The selected request is a GET request to `http://localhost:8082/bff-cajero/cuentas/retiros/c761ed6f-be5f-487c-bc2e-bb312322c6c5`. The response is a 200 OK status with a response time of 7 ms and a body size of 403 B. The response body is displayed in JSON format.

Query Params

Key	Value	Description
Key	Value	Description

Body

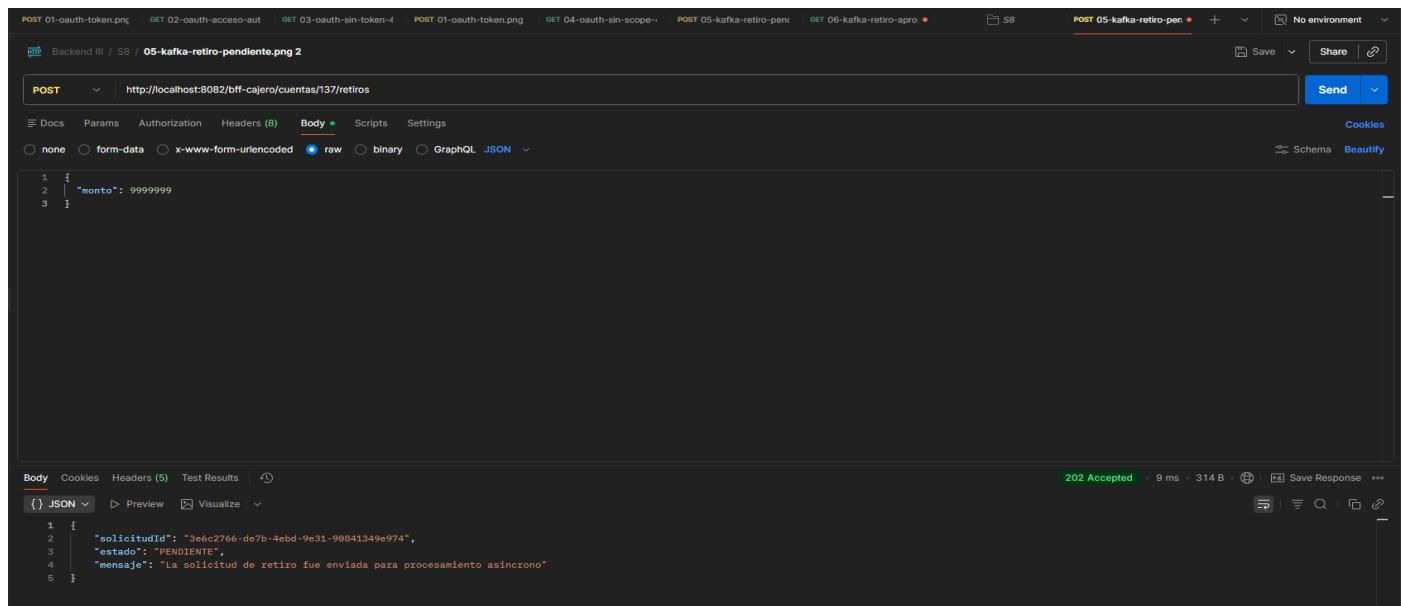
```
1 {
2   "solicitudId": "c761ed6f-be5f-487c-bc2e-bb312322c6c5",
3   "cuentaId": 137,
4   "estado": "APROBADO",
5   "monto": 50000,
6   "saldoInicial": 450000.00,
7   "saldoFinal": 400000.00,
8   "motivo": "Retiro procesado correctamente",
9   "fechaProcesamiento": "2026-09-30T14:35:29.754996325"
10 }
```

Resultado del retiro aprobado. Se observa el monto procesado, saldo inicial y saldo final, confirmando la actualización persistida.

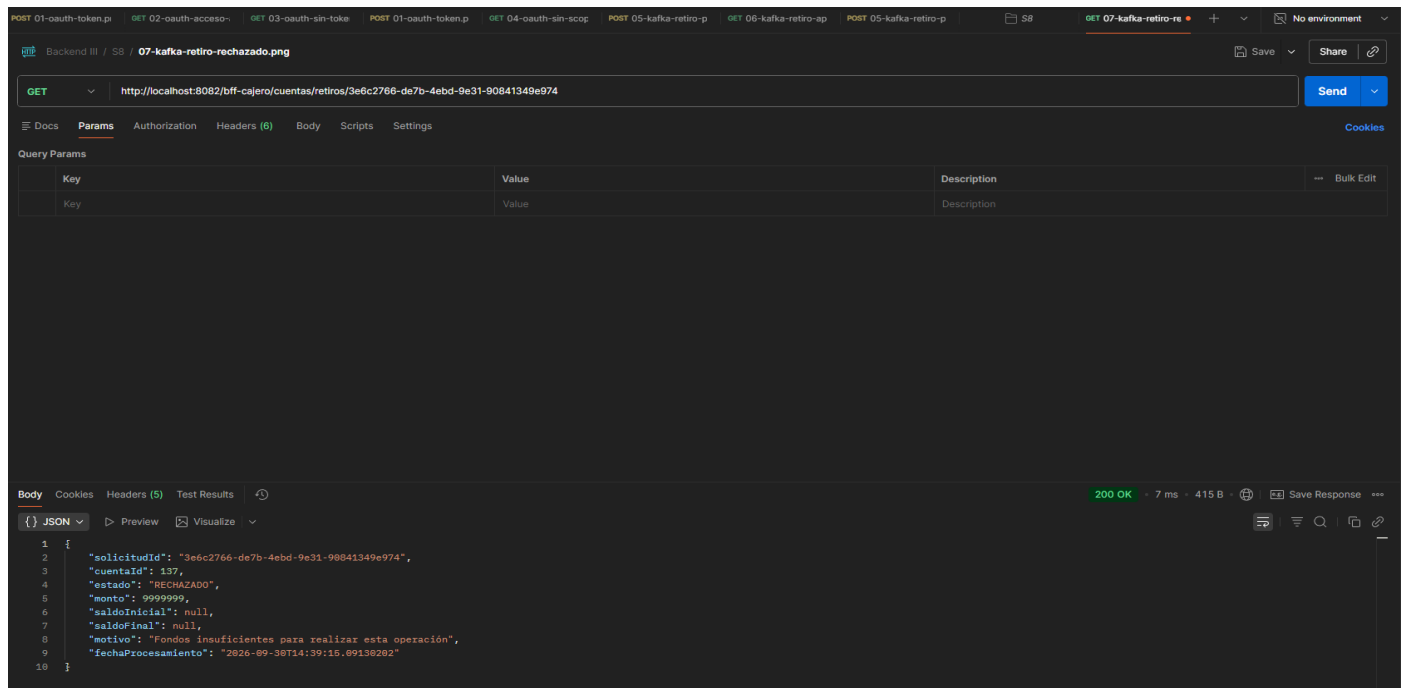
El flujo completo queda compuesto por publicación del evento desde BFF Cajeros, consumo en Banco Central, ejecución de la regla de negocio, publicación del resultado y consumo final en el BFF.

Apache Kafka - Retiro rechazado

Se realiza una segunda prueba solicitando un monto superior al saldo disponible. El BFF mantiene el comportamiento asíncrono y devuelve inicialmente HTTP 202 Accepted con estado PENDIENTE.



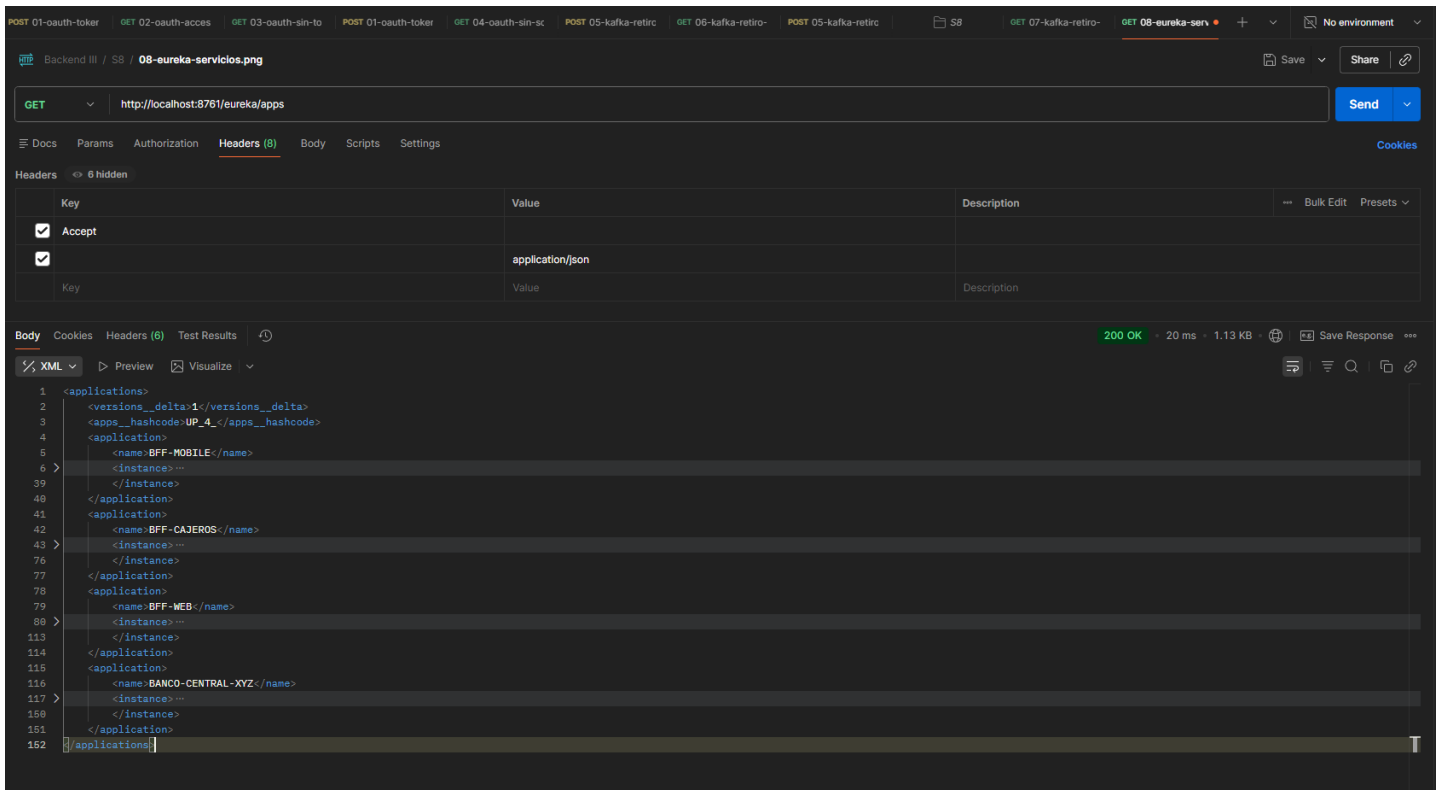
Solicitud de retiro por un monto superior al saldo disponible, aceptada para procesamiento as\u00edncrono.



Resultado RECHAZADO informado por el BFF. Banco Central indica como motivo: Fondos insuficientes para realizar esta operaci\u00f3n.

Eureka Server - Descubrimiento de servicios

Eureka Server se ejecuta en el puerto 8761. La consulta GET /eureka/apps permite verificar el registro de los microservicios que participan en la arquitectura distribuida.



The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** http://localhost:8761/eureka/apps
- Status:** 200 OK
- Response Time:** 20 ms
- Response Size:** 1.13 KB
- Headers:** 6 hidden
- Body:** XML

The XML response body is as follows:

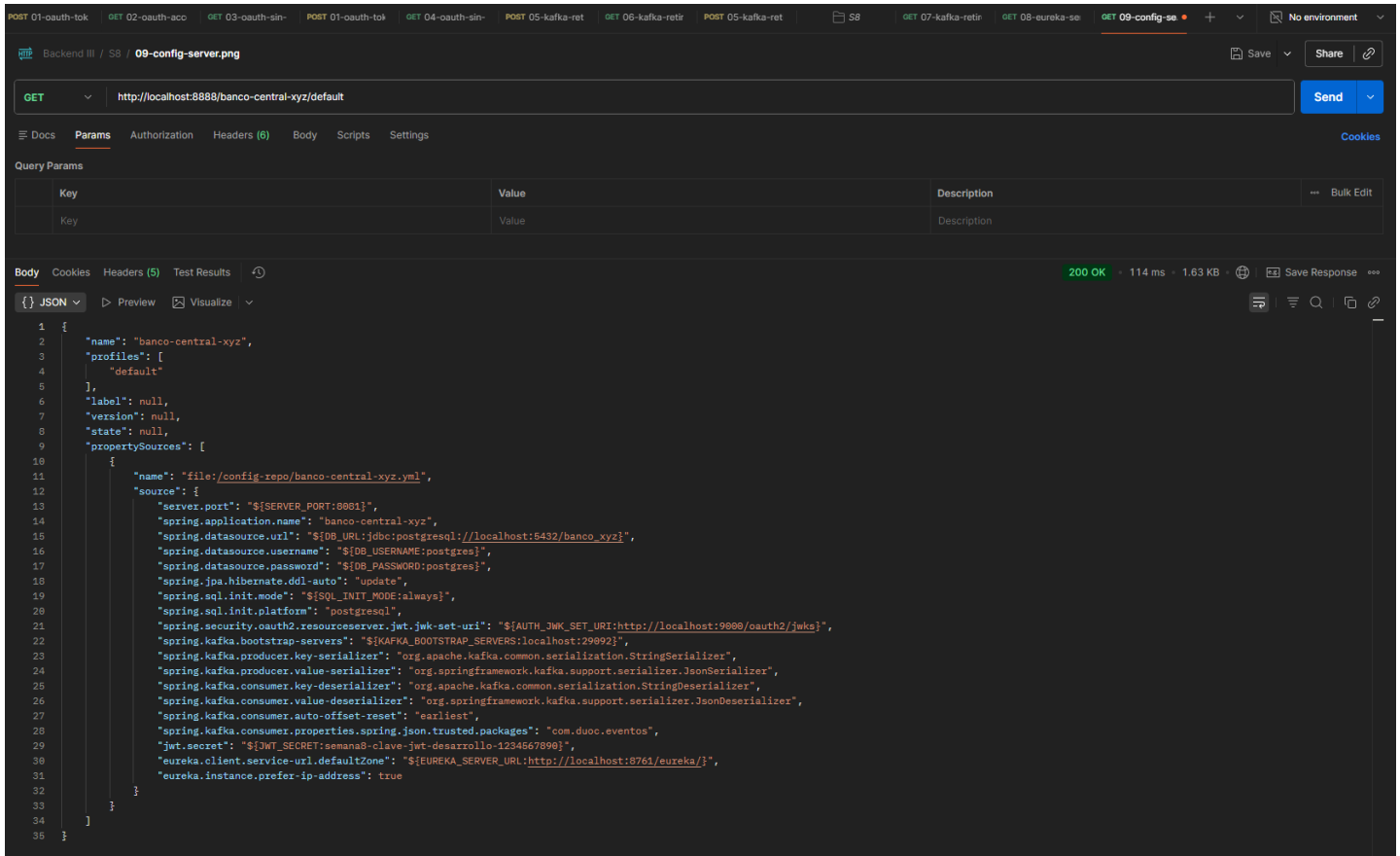
```
<?xml version='1.0' encoding='UTF-8'?>
<applications>
  <versions_delta>1</versions_delta>
  <apps_hashcode>UP_4_</apps_hashcode>
  <application>
    <name>BFF-MOBILE</name>
    <instance>...
  </instance>
</application>
  <application>
    <name>BFF-CAJEROS</name>
    <instance>...
  </instance>
</application>
  <application>
    <name>BFF-WEB</name>
    <instance>...
  </instance>
</application>
  <application>
    <name>BANCO-CENTRAL-XYZ</name>
    <instance>...
  </instance>
</application>
</applications>
```

Respuesta del registro de Eureka mostrando BFF-MOBILE, BFF-CAJEROS, BFF-WEB y BANCO-CENTRAL-XYZ registrados y disponibles.

El descubrimiento de servicios permite desacoplar las ubicaciones físicas de los microservicios y facilita su operación dentro del entorno Docker Compose.

Spring Cloud Config Server

Config Server se ejecuta en el puerto 8888 y centraliza la configuración del microservicio Banco Central. La consulta GET /banco-central-xyz/default devuelve las propiedades cargadas desde config-repo.



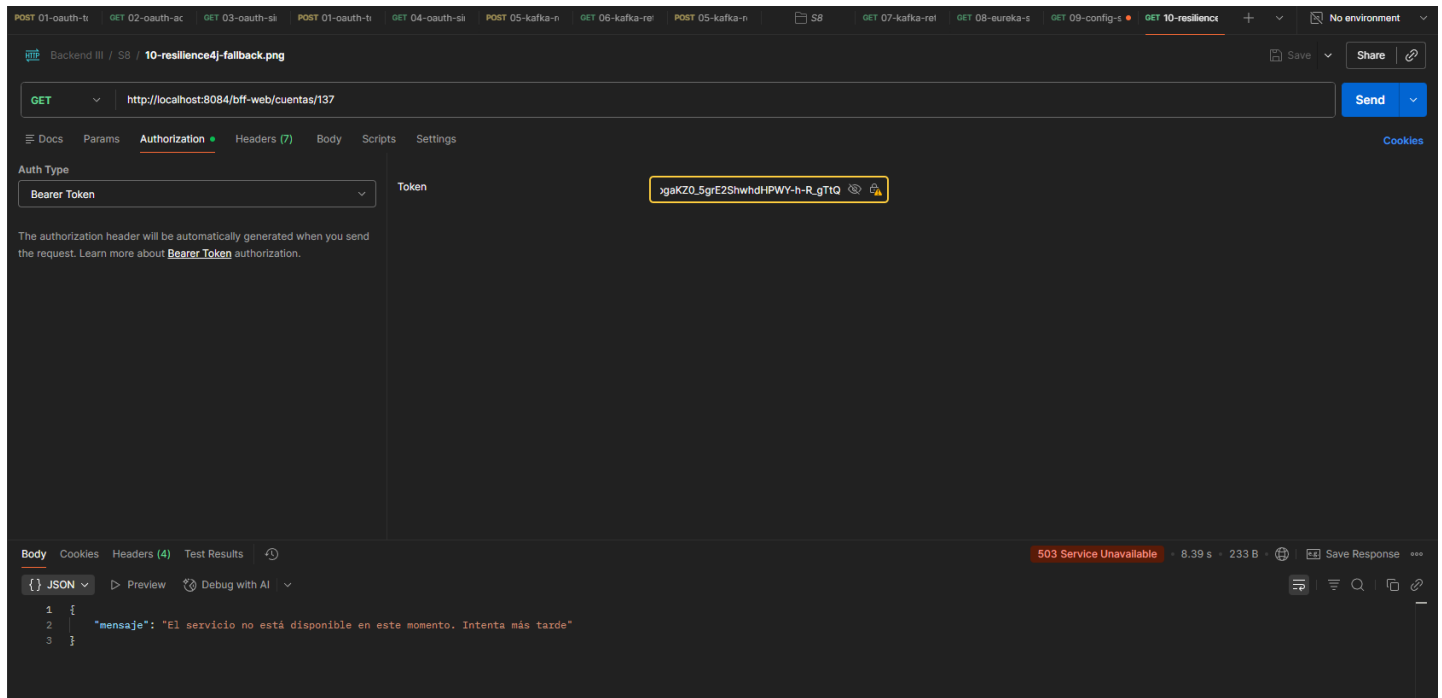
```
1 {
2   "name": "banco-central-xyz",
3   "profiles": [
4     "default"
5   ],
6   "label": null,
7   "version": null,
8   "state": null,
9   "propertySources": [
10    {
11      "name": "file:/config-repo/banco-central-xyz.yml",
12      "source": {
13        "server.port": "${SERVER_PORT:8881}",
14        "spring.application.name": "banco-central-xyz",
15        "spring.datasource.url": "${DB_URL:jdbc:postgresql://localhost:5432/banco_xyz}",
16        "spring.datasource.username": "${DB_USERNAME:postgres}",
17        "spring.datasource.password": "${DB_PASSWORD:postgres}",
18        "spring.jpa.hibernate.ddl-auto": "update",
19        "spring.sql.init.mode": "${SQL_INIT_MODE:always}",
20        "spring.sql.init.platform": "postgresql",
21        "spring.security.oauth2.resourceserver.jwt.jwt-set-uri": "${AUTH_JWK_SET_URI:http://localhost:9000/oauth2/jwks}",
22        "spring.kafka.bootstrap.servers": "${KAFKA_BOOTSTRAP_SERVERS:localhost:29092}",
23        "spring.kafka.producer.key-serializer": "org.apache.kafka.common.serialization.StringSerializer",
24        "spring.kafka.producer.value-serializer": "org.springframework.kafka.support.serializer.JsonSerializer",
25        "spring.kafka.consumer.key-deserializer": "org.apache.kafka.common.serialization.StringDeserializer",
26        "spring.kafka.consumer.value-deserializer": "org.springframework.kafka.support.serializer.JsonDeserializer",
27        "spring.kafka.consumer.auto-offset-reset": "earliest",
28        "spring.kafka.consumer.properties.spring.json.trusted.packages": "com.duoc.events",
29        "jwt.secret": "${JWT_SECRET:semana8-clave-jwt-desarrollo-1234567890}",
30        "eureka.client.service-url.defaultZone": "${EUREKA_SERVER_URL:http://localhost:8761/eureka/}",
31        "eureka.instance.prefer-ip-address": true
32      }
33    }
34  ]
35 }
```

Respuesta HTTP 200 del Config Server. Se observan propiedades de datasource, OAuth2 Resource Server, Kafka y Eureka centralizadas en banco-central-xyz.yml.

La configuración utiliza variables de entorno con valores de desarrollo, permitiendo ejecutar la misma imagen en distintos contextos sin modificar el código fuente.

Resilience4j - Fallback ante caída de Banco Central

Para comprobar la tolerancia a fallos se detiene temporalmente el contenedor banco-central y se ejecuta una petición desde BFF Web. Los clientes incorporan Circuit Breaker, Retry, Rate Limiter y métodos fallback.



Con Banco Central no disponible, BFF Web responde HTTP 503 Service Unavailable con un mensaje controlado en lugar de propagar una excepción sin manejo.

El tiempo de respuesta evidencia la aplicación de la política de reintentos antes de ejecutar el fallback. El servicio Banco Central se inicia nuevamente después de la prueba.



Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.